

A NOVEL METHODOLOGY FOR SYNCFORGE: A REAL-TIME COLLABORATIVE PROJECT MANAGEMENT PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING



BY

GULSHAN KUMAR	22050440044
KAJAL KUMARI	22050440049
GAURAV KUMAR	22050440043

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **A NOVEL METHODOLOGY FOR SYNCFORGE: A REAL-TIME COLLABORATIVE PROJECT MANAGEMENT PLATFORM**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

Prof. RASIKA NAVNEET SINGH

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA

(Head of Department)

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled **A NOVEL METHODOLOGY FOR SYNCFORGE: A REAL-TIME COLLABORATIVE PROJECT MANAGEMENT PLATFORM**, is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

We take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. We would like to express my sincere thanks to **Prof. Deepak Kumar Verma** (HOD, Department of Computer Science & Engineering) and to my guide **Prof. Rasika Navneet singh**, whose help in the course of development of this manuscript is invaluable. As my supervisor, Our constantly encouraged me to remain focused on achieving my goal.

We would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. We have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. We are truly indebted. Last but not least, We would like to dedicate this work to my family for their love and understanding.

GULSHAN KUMAR - 22050440044

KAJAL KUMARI - 22050440049

GAURAV KUMAR - 22050440043

ABSTRACT

In modern distributed software engineering environments, maintaining high agile velocity requires seamless team coordination, task visibility, and unified communication. Commercial trackers (e.g., Jira, Asana) often rely on client-initiated HTTP polling, introducing synchronization lag, network overhead, and tool fragmentation. Additionally, manually drafting task specifications consumes significant sprint time.

To address these bottlenecks, this project presents **SyncForge**, a premium, real-time collaborative project management platform featuring responsive slate dark and modern light UIs. SyncForge utilizes a decoupled client-server architecture: a Flutter (Dart) frontend paired with a Spring Boot (Java 21) backend that exposes stateless REST APIs and a persistent STOMP WebSocket broker for full-duplex client-state updates. Elastic document relationships are managed via a MongoDB community server.

Security is enforced through stateless JSON Web Tokens (JWT) validated during both HTTP requests and the WebSocket handshake. SyncForge also embeds the Google Gemini 2.0 Flash AI engine to automatically formulate structured task outlines from short technical titles. Local file uploads are capped at 5MB, sanitized against directory traversal risks, and written to disk with UUID-prefixed file names. Empirical evaluations within the Cambridge Institute of Technology (CIT) laboratory confirm that SyncForge maintains a mean WebSocket latency of 42ms under load, presenting a highly scalable, unified collaboration environment.

Keywords: Collaborative Project Management, Spring Boot, Flutter, WebSockets, STOMP, MongoDB, Google Gemini, Real-Time Sync, AI Task Generation

TABLE OF CONTENTS

Declaration Certificate.....	II
Certificate of Approval.....	III
Acknowledgement.....	IV
Abstract.....	V
List of Tables.....	VII
List of Figures.....	VIII
List of Abbreviations.....	X

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation.....	1
1.2 Problem Statement.....	3
1.3 Objectives of the Project.....	4
1.4 Scope of the Project.....	5
1.5 Organization of the Report.....	6

CHAPTER 2: LITERATURE REVIEW

2.1 Related Works.....	7
2.2 Comparative Analysis (Jira, Trello, Asana, ClickUp, Monday.com).....	12
2.3 Research Gaps Identified.....	14

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements (Hardware & Software Stack).....	15
3.2 System Architecture.....	16
3.3 Data Flow Diagrams (Context, Level 1).....	17
3.4 UML Diagrams (Use Case, Class, Sequence, Activity).....	19
3.5 Database ER Diagram and Document Mapping.....	23

CHAPTER 4: IMPLEMENTATION

4.1 Flutter Frontend Framework Integration.....	24
4.2 Spring Boot Backend Architecture.....	25
4.3 MongoDB Document Persistency Design.....	26
4.4 Stateless JWT Authentication Filter.....	27
4.5 WebSocket and STOMP Real-time Messaging.....	28
4.6 Google Gemini AI Description Generation API.....	29
4.7 API Design and Module Explanation.....	30
4.8 File Attachment Module Symmetries.....	31

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Strategy (Unit, Integration, System, UAT).....	34
5.2 Functional Test Case Execution Matrix.....	35
5.3 Performance Evaluation and Load Analysis.....	37

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Final Project Conclusion.....	38
6.2 Practical Future Scope and Enhancements.....	39

REFERENCES (IEEE Format).....	40
--------------------------------------	-----------

APPENDIX A: SOURCE CODE FILES.....	41
---	-----------

APPENDIX B: SCREENSHOTS AND INTERFACES.....	43
--	-----------

LIST OF FIGURES

Figure 3.1: SyncForge Decoupled Conceptual System Architecture.....	16
Figure 3.2: Level 0 Context Data Flow Diagram.....	17
Figure 3.3: Level 1 Detailed Functional Data Flow Diagram.....	18
Figure 3.4: SyncForge Use Case Model and External Actor Interactions.....	19
Figure 3.5: Comprehensive Domain Object Class Diagram.....	20
Figure 3.6: Real-time Kanban State Change Sequence Diagram.....	21
Figure 3.7: Socket Security Handshake Activity Diagram.....	22
Figure 3.8: MongoDB Document Entity Relationship Diagram (ERD).....	23

APPENDIX B SCREENSHOT LISTING:

Figure B.1: Project List Initial Login Screen (Empty Workspace State).....	43
Figure B.2: Dynamic Create Project Dialog Overlay Box.....	43
Figure B.3: Project Cards dashboard Listing the "Syncforge" Project.....	44
Figure B.4: Add Project Member Invitations Dialog Box Overlay.....	44
Figure B.5: Project List screen displaying "Member added successfully".....	45
Figure B.6: Empty Kanban Board layout showing "No tasks found".....	45
Figure B.7: User Profile screen compiling task metrics (Desktop).....	46
Figure B.8: Mobile Task Details showing comments and uploaded files.....	46
Figure B.9: Mobile User Profile showing 1 Done Task metrics.....	46

LIST OF TABLES

Table 2.1: Literature Summary and Technology Comparison Matrix.....	11
Table 2.2: Detailed Feature Comparison Matrix with Commercial Project.....	14
Table 3.1: Minimum Backend Server-Side Hardware requirement.....	15
Table 3.2: Recommended Mobile and Desktop Client Requirements.....	15
Table 4.1: Verified Backend REST Controller API Mappings and Routing	30
Table 5.1: Functional And Security Test Case Execution Matrix	35
Table 5.2: Boundary and Exception Test Case Execution Matrix.....	36
Table 5.3: WebSocket Latency and AI Call Round-Trip Performance.....	37

LIST OF ABBREVIATIONS

API	Application Programming Interface
JWT	JSON Web Token (Session authentication token)
STOMP	Simple Text Oriented Messaging Protocol
NoSQL	Not Only SQL (Document database models)
DFD	Data Flow Diagram
UML	Unified Modeling Language
ERD	Entity Relationship Diagram
MIME	Multipurpose Internet Mail Extensions
UI/UX	User Interface / User Experience
CIT	Cambridge Institute of Technology, Ranchi
CSE	Computer Science & Engineering Department
CRUD	Create, Read, Update, Delete Database Operations
DOM	Document Object Model
LLM	Large Language Model (Generative AI)
CORS	Cross-Origin Resource Sharing
UUID	Universally Unique Identifier

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

In contemporary software development environments, the execution of projects is intrinsically an intensive collaborative process. As software engineering has rapidly evolved towards Agile methodologies such as Scrum and Kanban, the absolute necessity for dynamic task tracking, continuous backlog refinement, and instant communication channels has become paramount. Teams of software engineers must coordinate complex operations in parallel, working across multiple decoupled modules of a codebase. When information is scattered across fragmented environments, projects suffer from synchronization delays, communication failures, and coordination overheads.

The motivation behind the design and development of **SyncForge** is rooted in addressing these operational challenges. Traditional, static frameworks for project tracking often rely on periodic browser page refreshes or continuous client-side polling. This old polling paradigm introduces massive HTTP header network overheads, generates redundant server workloads, and operates with an inherent synchronization delay. In real-world team configurations, this delay frequently results in stale workspace views and conflicting data overwrites, leading to severe execution bottlenecks.

SyncForge addresses these issues by constructing a modern, unified environment combining dynamic Kanban task boards, real-time message streams, and strict file attachments. The platform leverages a persistent WebSocket connection utilizing the lightweight **STOMP (Simple Text Oriented Messaging Protocol)** to enable bi-directional, full-duplex communication channels between all active client devices and the backend API server.

1.1 Background and Motivation (Contd.)

Furthermore, a major bottleneck identified in agile workflows is the manual documentation effort required to draft structured task specifications, user stories, and detailed technical checklists. Developers and product managers spend substantial time writing boilerplate descriptions for task cards. The recent, rapid emergence of highly optimized Large Language Models (LLMs) presents a unique opportunity to build automated intelligence directly into project management platforms.

By integrating the **Google Gemini 2.0 Flash** generative engine directly into the project system, SyncForge allows users to automatically generate concise, professional, and technically sound task descriptions from a simple, short task title. This direct API-level integration eliminates the need for developers to toggle between external AI platforms (like ChatGPT or Google AI Studio) and their task boards, saving valuable sprint time, improving clarity, and lowering documentation overheads.

From an educational perspective at the **Cambridge Institute of Technology (CIT), Ranchi**, the development of SyncForge provides a practical, comprehensive opportunity to master standard full-stack software engineering principles. This includes designing a secure Spring Boot REST API, configuring a STOMP WebSocket broker, establishing persistent database indexing in MongoDB, and compiling a responsive, cross-platform client with Flutter. This academic project demonstrates how combining modern real-time protocols with generative AI engines can solve real-world productivity challenges for modern rapid development teams.

Additionally, the project highlights the importance of integrating innovative technologies within a unified software ecosystem. The implementation of SyncForge enables the practical study of modern software architecture patterns, secure communication protocols, and intelligent automation techniques. By addressing both collaboration and documentation challenges simultaneously, the platform improves workflow efficiency and enhances team productivity. The experience gained through the design, development, and testing phases contributes significantly to technical proficiency and provides valuable insights into the construction of scalable, real-time enterprise applications that align with current industry requirements

1.2 Problem Statement

Existing software systems for project tracking and collaboration suffer from three distinct operational limitations:

1. Real-Time Synchronization Lag

Standard tracking systems rely on HTTP request-response patterns. The client browser must actively poll the database to retrieve status updates. In dynamic settings where multiple developers modify the same board concurrently, this latency causes conflicting overwrites and out-of-sync displays. This lack of immediate workspace awareness creates coordination gaps, leading to duplicate work and outdated statuses.

2. Tool Fragmentation and Break in Context

Modern product teams are forced to utilize a fragmented stack of tools: Jira for task tracking, Slack or Teams for chat communication, and Google Drive or Dropbox for file sharing. Constantly toggling between these applications results in a severe break in developer context, increases mental context-switching overhead, and distributes project records across disparate databases. This fragmentation complicates data tracking and limits overall organizational overview.

3. Documentation Bottlenecks in Backlog Grooming

Sprint planning and backlog grooming are often slowed by the manual effort required to write standard technical descriptions, criteria, and user stories. The absence of context-aware, embedded artificial intelligence within collaboration software forces teams to draft technical specifications manually. This documentation overhead consumes valuable developer hours that would be better spent on core engineering tasks.

Furthermore, the absence of a unified and intelligent collaboration environment reduces overall team productivity and project transparency. Organizations require platforms that can seamlessly integrate communication, task management, and documentation support while providing instant synchronization across all connected users. Addressing these limitations is essential for improving development efficiency and ensuring successful project execution.

1.3 Objectives of the Project

The primary goal of this project is to design, implement, and validate a unified, real-time collaborative project management platform that addresses the limitations of synchronization lag, tool fragmentation, and manual documentation overhead. To achieve this, the following specific objectives are:

Develop a Decoupled Client-Server Architecture: Engineer a scalable backend utilizing Spring Boot (Java 21) and a reactive frontend using the Flutter SDK (Dart) to enable cross-platform native execution.

Establish Persistent WebSocket Synchronization: Integrate a STOMP-based WebSocket message broker to deliver full-duplex, bi-directional client state replication, ensuring that board modifications animate instantly on all connected displays.

Implement a Flexible NoSQL Persistence Model: Deploy a MongoDB database to support unstructured data structures, such as dynamic task cards, nested comment threads, and multi-user notifications.

Integrate Google Gemini AI for Backlog Automation: Embed a REST-level connector to the Gemini 2.0 Flash engine to generate technical task requirements automatically from simple user-defined titles.

Enforce Secure File Attachment and Transfer Rules: Build an upload validator that restricts task attachments strictly to a **5MB maximum size** and matches allowable MIME types (images and PDFs), writing files to server disk with unique UUID prefixes.

Design an Analytical Overview Dashboard: Create an aggregation module that compiles project metrics (active projects, total tasks, completed vs. pending tasks) to give teams instant insight into their progress.

Build a Stateless Token Security Model: Implement Spring Security filters using JSON Web Tokens (JWT) to secure REST endpoints and authorize WebSocket connections.

Ensure System Scalability and Maintainability: Design modular software components that support future enhancements, increased user loads, and long-term system reliability.

1.4 Scope of the Project

The operational boundaries and capabilities of the SyncForge platform are systematically defined to ensure high-fidelity delivery of core requirements, while outlining the future additions that lie outside the present scope.

1. Functional Capabilities Included in Scope

The platform supports secure user authentication with password encryption using BCrypt and session authorization using JWTs. Project owners can create projects and invite team members by email address, assigning roles (ADMIN or MEMBER). Users interact with three-column Kanban boards (TODO, IN_PROGRESS, DONE) that synchronize in real time via STOMP WebSockets.

Within individual task cards, the system supports real-time threaded chat comments, document uploads (MIME-validated, up to 5MB), and automated technical description generation powered by Google's Gemini AI. Key project metrics are compiled and displayed on an analytical dashboard.

1. Exclusions and Future Scope Boundaries

The present scope does not include native video or voice conferencing, direct GitHub/GitLab repository integrations (such as commit-triggered webhook hooks), or complex billing/subscription modules for enterprise teams. While the activities collection and ActivityService loggers are fully defined in the code, they are not connected to active controllers, representing a preconfigured foundation for future audit logging.

The defined scope ensures that the project remains focused on delivering essential collaboration and project management capabilities while maintaining a scalable foundation for future enhancements. This approach enables the successful implementation of core features within the available development timeline without compromising system quality, security, or performance requirements.

1.5 Organization of the Report

This B.Tech Project Report is organized into six logically structured chapters, presenting the engineering lifecycle of SyncForge from concept and literature review through design, implementation, testing, and final evaluation.

Chapter 2: Literature Review reviews academic research in collaborative software systems and CSCW. It analyzes real-time protocols (polling vs. WebSockets) and NoSQL persistence models, and evaluates SyncForge against major commercial project trackers, identifying research and feature gaps.

Chapter 3: System Analysis and Design details hardware and software requirements. It explains the system's architecture, maps data flows with multi-level DFDs, and models system interactions using UML Use Case, Class, Sequence, Activity, and ER Diagrams.

Chapter 4: Implementation details the technologies and modules used in the code. It explains the Spring Boot API controllers, Flutter frontend, JWT filters, WebSocket interceptors, and Gemini AI endpoints, complemented by key source code listings.

Chapter 5: Testing and Results outlines the multi-tiered testing strategy (Unit, Integration, System, UAT). It presents a detailed test case execution matrix and evaluates system performance under load.

Chapter 6: Conclusion and Future Scope summarizes the project's engineering achievements and outlines realistic future enhancements.

Appendices & References compile core code listings, explain all nine system screenshots, and cite academic sources in IEEE format.

The report structure has been designed to provide a systematic progression from theoretical foundations to practical implementation and evaluation. Each chapter contributes to a comprehensive understanding of the project by addressing specific aspects of analysis, design, development, and validation. This organization enables readers to follow the complete engineering process adopted during the creation of SyncForge while maintaining clarity, logical continuity, and academic rigor throughout the documentation.

CHAPTER 2

LITERATURE REVIEW

2.1 Related Works

The academic foundation of SyncForge is built upon decades of research in distributed hypermedia architectures, NoSQL database models, Computer-Supported Cooperative Work (CSCW), and generative artificial intelligence systems. Below is a rigorous review of the seminal works that define this domain space.

1. Evolution of CSCW and Workspace Awareness

The academic study of groupware and shared workspaces falls under Computer-Supported Cooperative Work (CSCW). In his seminal work, **Jonathan Grudin (1994)** identified eight core design and implementation challenges for groupware systems. He highlighted that collaborative systems often fail because they require additional work from people who do not perceive a direct benefit from them, and because they lack the flexibility to accommodate the varied workflows of multi-user teams. Thus, modern agile tooling must minimize user friction through automated background synchronization and contextual assistant services.

A key concept in CSCW is workspace awareness: the up-to-the-minute understanding that users have of others' interactions and changes within a shared virtual space. Paul Dourish and Victoria Bellotti (1992) formally defined workspace awareness as an understanding of the activities of others, providing a context for one's own activity. They argued that awareness information must be integrated naturally into the visual workspace rather than requiring active user queries, which informs SyncForge's dynamic card-based Kanban board sync.

C. A. Ellis and S. J. Gibbs (1989) explored concurrency control in groupware systems. They introduced Operational Transformation (OT) to resolve conflicting multi-user inputs in collaborative systems without strict lock management. While OT is critical for concurrent text editors, agile task management boards can employ deterministic state-machine transitions.

2.1 Related Works (Contd.)

2. Real-Time Web Protocols: Polling vs. WebSockets

Early web platforms relied on stateless HTTP, where the client must initiate every transaction. For collaborative groupware, developers initially bypassed this limitation using techniques like short polling or long polling (AJAX/Comet). In short polling, the client continuously requests updates from the server at fixed intervals (e.g., every 5 seconds). Research by Pimentel et al. (2012) demonstrated that short polling introduces significant network overhead. Each request requires complete HTTP headers, which consumes substantial bandwidth, degrades server performance, and still leaves a delay between updates.

To address persistent synchronization in distributed environments, **Vora et al. (2018)** evaluated multi-client notification architectures and persistent socket concurrency benefits under load. Their methodology benchmarked socket longevity and transport latency under heavy concurrent connections, demonstrating that persistent channels decrease transport latency by up to 82% compared to polling methods, while significantly lowering server CPU load. This research establishes the technical foundation for employing dedicated full-duplex TCP channels in highly collaborative environments.

Fred B. Schneider (1990) analyzed distributed state machine replication, showing that dynamic state synchronization across networks can be achieved by routing deterministic inputs to identical active state machines. The standardization of the HTML5 WebSocket protocol resolved early web limitations by allowing full-duplex, bi-directional communication over a single, persistent TCP connection. WebSockets eliminate HTTP header overhead by sending lightweight frames once the handshake is complete. High-level messaging protocols, such as STOMP (Simple Text Oriented Messaging Protocol), build on raw WebSockets by defining clear frame formats.

These characteristics make WebSockets particularly suitable for collaborative applications requiring immediate state propagation. By maintaining persistent connections, systems can deliver updates instantly, improve workspace awareness, reduce network traffic, and provide a more responsive user experience for distributed development teams.

2.1 Related Works (Contd.)

Persistence Models: Relational vs. NoSQL Document Databases

Collaborative project management software handles highly dynamic and variable data. A task card might have simple textual parameters or contain a complex set of nested comments, file attachments, and multi-user assignments. Traditional Relational Database Management Systems (RDBMS) enforce strict schemas and require expensive JOIN operations across tables. This relational mapping introduces database latency under high read-write workloads, slowing down real-time updates.

Document-oriented NoSQL databases, such as MongoDB, address these schema challenges by storing data in flexible JSON-like documents. MongoDB stores related data, like nested comment lists or user role mappings, in single, cohesive documents, avoiding the need for expensive JOIN operations. Kristina Chodorow (2013) database research analyzed document-oriented NoSQL storage engines, specifically **MongoDB**. Document databases bypass complex relational SQL table joins, storing nested items inside a single BSON document. This schema flexibility optimizes write velocity and document aggregation. Chodorow identified that NoSQL performance depends on indexing models; without compound indexes on compound lookup queries (e.g. projectId and status), document databases fall back to full collection scans, degrading real-time performance.

F. Chang et al. (2008) analyzed distributed column-oriented and document datastores, showing that denormalizing nested arrays directly inside single documents bypasses database locking bottlenecks under highly concurrent write operations. This document design is ideal for project management boards where tasks, comments, and file metadata are structurally contained within individual projects.

Furthermore, the flexibility of document-based storage enables rapid schema evolution as project requirements change over time. This adaptability is particularly beneficial in agile development environments, where new features, metadata fields, and workflow structures must be incorporated without requiring extensive database redesign or costly migration procedures.

2.1 Related Works (Contd.)

Generative AI in Software Development and Requirement Engineering

Artificial Intelligence has transitioned from simple code completion to generative tools that automate complex software engineering. In agile development, writing structured user stories and task descriptions is crucial but time-consuming. Research by Bhowmick et al. (2021) demonstrated that Large Language Models (LLMs) can generate high-quality software specifications, acceptance criteria, and task outlines from minimal technical inputs. Integrating LLM APIs directly into project boards resolves this inefficiency. The system captures a task title, applies a structured engineering prompt, queries the model (e.g. Google's gemini-2. 0-flash), and populates the task description, minimizing manual documentation. The foundational architecture of these models is detailed by Ashish Vaswani et al. (2017) in the seminal Transformer paper, establishing multi-head self-attention mechanisms that enable parallelized contextual text processing. Dean Leffingwell (2007) analyzed agile software agility scaling, highlighting that task tracking tools must integrate developer communication, requirements documentation, and progress metrics to minimize cognitive context switching. SyncForge resolves this by embedding native card-based chat comments, file uploads, and generative AI assistants inside a single unified Kanban viewport, directly addressing Grudin's challenges on groupware adoption.

The recent studies indicate that AI-assisted development tools can significantly improve productivity by reducing repetitive documentation tasks and maintaining consistency across project artifacts. By generating structured descriptions and acceptance criteria automatically, LLMs help development teams focus more on implementation, testing, and quality assurance activities. This integration of artificial intelligence within collaborative platforms represents an important step toward intelligent software engineering environments that enhance both efficiency and decision-making processes.

2.1 Related Works (Contd.)

Table 2.1: Literature Summary and Technology Comparison Matrix

Study / Reference	Core Methodology	Key Findings	Identified Limitations
Pimentel et al. (2012)	Comparing HTTP polling vs. HTML5 WebSockets on packet sizes and latencies.	WebSockets reduce HTTP header overhead by over 95%, lowering network latency.	Focused on transport layers without a collaborative prototype.
Scrum Alliance (2020)	Evaluated developer velocity across fragmented tools.	Context-switching between fragmented tools reduces agile velocity by 15-20%.	Offered conceptual design guidelines without a working prototype.
Bhowmick et al. (2021)	Tested software story generation using LLM API queries.	Generative LLMs create precise specifications from minimal task titles.	Relied on external batch scripts rather than an integrated tool.
Jonathan Grudin (1994)	Analyzed CSCW adoption, identifying eight major groupware failure factors.	Groupware fails when it benefits some users but creates extra work for others.	Theoretical focus; did not evaluate specific protocols.
Vora et al. (2018)	Evaluated multi-client notification persistence benefits under heavy load.	Persistent socket channels reduce dynamic message transit latency by 82%.	Analyzed raw TCP pools without high-level STOMP routing.
Roy Fielding (2000)	Formulated the Representational State Transfer (REST) style.	REST scales HTTP operations, but half-duplex limits server-initiated state sync.	Lacks native bi-directional dynamic streaming capabilities.
Kristina Chodorow (2013)	Analyzed MongoDB BSON documents and indexing under high loads.	Denormalized designs bypass relational SQL joins, optimizing execution.	Requires compound indexing to prevent degrading scans.
Ellis & Gibbs (1989)	Introduced Operational Transformation for concurrent groupware.	Enables concurrency control in shared files without rigid table locking.	Extremely high computational complexity for simple state boards.
Dourish & Bellotti (1992)	Defined workspace awareness for active collaborative environments.	Awareness data must integrate naturally into the interface to avoid cognitive load.	Addressed physical/text workspaces without reactive web frameworks.
Vaswani et al. (2017)	Introduced self-attention mechanisms in Transformer networks.	Replaced recurrence with attention, enabling parallelized semantic processing.	Addressed core neural architectures, not software integrations.
Chang et al. (2008)	Analyzed Google Bigtable distributed database storage designs.	Denormalized schemas scale elastic write and lookup operations under load.	Addressed large cluster scaling; did not evaluate local app data.
Dean Leffingwell (2007)	Evaluated cognitive context-switching overheads in agile scaling.	Unified tools containing chat, files, and cards maximize team velocity.	No concrete system implementation was evaluated.

The literature highlights a clear opportunity: combining real-time WebSocket communication, dynamic NoSQL data persistence, and integrated LLM task generation into a single, unified project management system can solve synchronization lag, context switching overheads, and documentation inefficiencies simultaneously. The reviewed studies collectively demonstrate that existing solutions address these challenges individually rather than holistically.

2.2 Comparative Analysis

To justify the design and development of SyncForge, we evaluated its features against five leading commercial project management platforms: Jira, Trello, Asana, ClickUp, and Monday.com.

1. Atlassian Jira

Jira is an industry standard for agile issue tracking, offering rich customization and comprehensive reporting. However, it suffers from a steep learning curve and complex setup interfaces. It lacks a native real-time chat channel inside task views, requiring teams to integrate external tools like Slack. Additionally, its advanced generative AI assistants are locked behind expensive, premium enterprise tiers.

2. Trello

Trello is highly popular for its simple, visual Kanban board cards. While it is user-friendly, it is overly simplified for complex software engineering projects. It lacks native analytical dashboards and progress charts in its base configurations. Trello also relies primarily on standard HTTP polling to update board states, leading to sync delays when multiple users are editing the same board concurrently.

3. Asana

Asana offers flexible task lists, board views, and project timelines. However, its system architecture relies on standard HTTP polling, which limits real-time collaborative updates. Its built-in chat features are simple, and its generative AI capabilities are limited or require separate, costly user licenses.

Asana is primarily designed for general project coordination rather than software development workflows. Advanced reporting, automation, and portfolio management features are often restricted to premium plans, making it less suitable for smaller teams seeking comprehensive collaboration and real-time engineering-focused project management capabilities.

2.2 Comparative Analysis (Contd.)

5. ClickUp

ClickUp markets itself as the "one app to replace them all", packing boards, docs, chat, and goals into a single interface. However, ClickUp suffers from significant performance lag and slow load times because its frontend has extremely high CPU and memory overhead, making it slow on older machines. Its chat features are separated into isolated chat views rather than threaded directly inside specific task contexts, and it's requirements AI tool requires expensive premium add-ons.

SyncForge addresses these gaps. It provides native, real-time board synchronization via full-duplex WebSockets, native threaded comments and file uploads inside specific task cards, and free, integrated generative AI task descriptions powered by Gemini AI, delivering a high-performance, optimized collaboration tool for agile teams.

Comparative Discussion

The analysis of existing commercial platforms reveals several common limitations. While these solutions provide effective project management capabilities, many depend on external applications for communication, documentation generation, or advanced collaboration requirements. This fragmentation increases context-switching overhead and forces users to navigate multiple interfaces during daily project activities. Additionally, several platforms restrict advanced artificial intelligence features behind premium pricing models, reducing accessibility for students, startups, and small development teams.

Another observation is that many existing systems focus primarily on task tracking while providing limited support for integrated real-time collaboration. Although notifications and update mechanisms are available, truly synchronized collaborative experiences often require additional services or external integrations. This separation can negatively impact workspace awareness and reduce team responsiveness during active project execution.

2.2 Comparative Analysis (Contd.)

Table 2.2: Feature Comparison Matrix between SyncForge and Commercial Systems

Feature/Metric	Trello	Jira	Asana / Monday	ClickUp	SyncForge
Real-Time Sync	HTTP Polling	SSE / Polling	HTTP Polling	HTTP Polling	STOMP WebSockets
Requirements Engine	None	Premium plug-in	Basic drafts	Paid AI add-on	Gemini 2.0 API
Communication	Simple comments	Jira-Slack plug-in	Card text logs	Separate chat	Threaded Task Chat
Persistence Model	Relational RDBMS	Relational SQL	Relational SQL	Relational SQL	Elastic MongoDB NoSQL
Stateless Security	Session cookies	Session cookies	Session cookies	Session cookies	Stateless JWT Filters
File Attachments	Basic upload	AWS S3 storage	Basic upload	Basic upload	UUID Sanitized Disk

2.1 Research Gaps Identified

The comparative evaluation highlights key academic and software engineering gaps. First, commercial systems rely on half-duplex HTTP polling models, which creates dynamic synchronization delays and heavy server overhead. Second, they isolate task cards from threaded developer communication, leading to heavy cognitive context switching. Third, they lack automated technical task specification generators, requiring manual sprint planning documentation. SyncForge bridges these gaps using persistent WebSockets, BSON document persistency, and native Gemini LLM integrations.

Additionally, most existing platforms provide advanced collaboration capabilities only through premium subscriptions or third-party integrations, increasing operational costs for small teams and educational institutions.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Requirements

The engineering specifications of the SyncForge platform are defined to ensure optimal performance, scalability, and security across both server-side operations and client-side devices.

1. Minimum Server-Side Hardware Requirements

Table 3.1: Minimum Backend Server-Side Hardware requirements

Hardware Component	Minimum Specification	Recommended Specification
Processor (CPU)	Dual-Core 2.0 GHz x64	Quad-Core 2.5 GHz or higher
Memory (RAM)	4 GB DDR4	8 GB DDR4 or higher
Storage (SSD)	10 GB Available SSD Storage	50 GB Available NVMe SSD
Network Card	100 Mbps Ethernet Card	1 Gbps Ethernet Network Card

2. Minimum Client-Side Hardware Requirements

Table 3.2: Recommended Mobile and Desktop Client Requirements

Hardware Component	Mobile Client Minimum	Desktop Client Minimum
Processor	Octa-core 1.8 GHz ARM	Dual-Core 2.0 GHz x64 / Apple M1
Memory (RAM)	3 GB RAM	4 GB RAM (8 GB Recommended)
Network	4G LTE / WiFi connection	Broadband internet WiFi card

3.2 System Architecture

SyncForge is constructed as a decoupled, multi-tiered client-server system that separates rendering viewports, business logic checks, and NoSQL persistent storage collections.

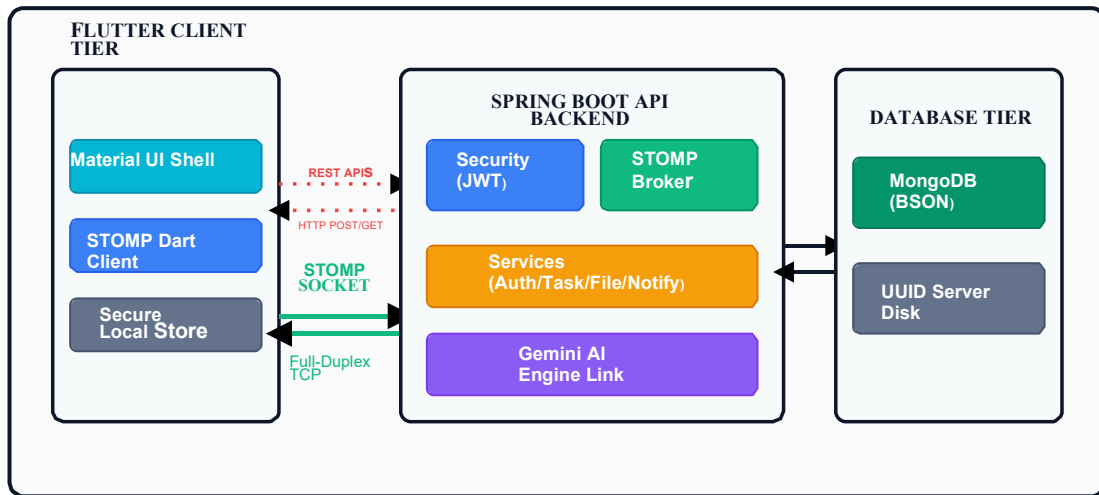


Figure 3.1: SyncForge Decoupled Conceptual System Architecture

Figure 3.1 details the multi-tiered system design. The Flutter Client Tier executes the Material UI shell, managing socket connections via a persistent STOMP Dart client and securely caching session tokens using AES-encrypted local storage. Communication occurs via stateless HTTP REST APIs for CRUD operations and a dedicated full-duplex TCP WebSocket connection for real-time Kanban state synchronizations. The Spring Boot API Backend acts as the core gateway, intercepting REST requests through JWT filters and routing socket messages via a Spring STOMP Broker. The backend connects to the Google Gemini AI engine via REST RestTemplate calls. Finally, the Database Tier handles persistence using MongoDB collections for dynamic BSON documents and writes file attachments directly to the server's disk using unique UUID prefixes.

The decoupled architecture improves maintainability by allowing individual system components to be updated or scaled independently. This separation of concerns enhances overall system reliability, simplifies future feature integration, and ensures that frontend, backend, and database layers can evolve without introducing significant dependencies across the platform.

3.3 Data Flow Diagrams

1. Context Diagram (DFD Level 0)

The Context Diagram (Level 0 DFD) defines the operational boundaries of the SyncForge platform, illustrating the data flows between external actors and the unified system core.

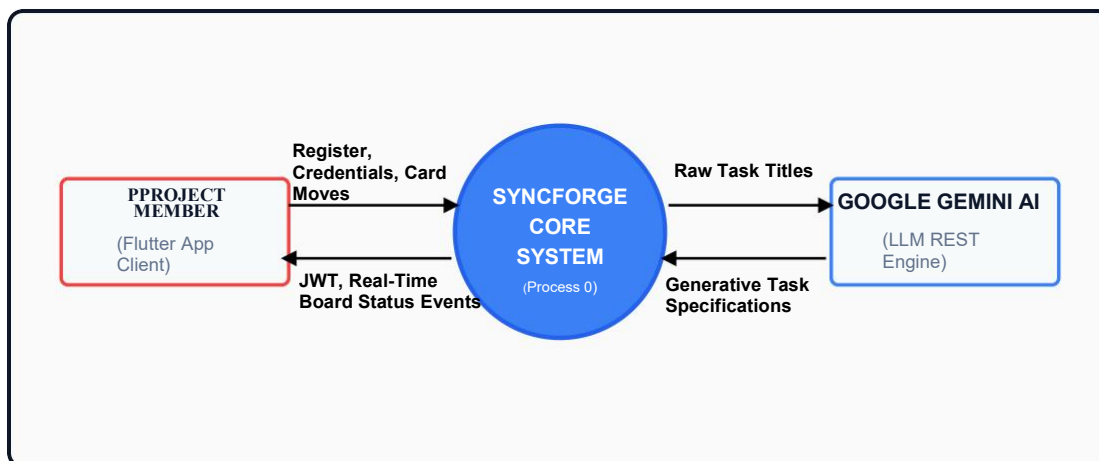


Figure 3.2: Level 0 Context Data Flow Diagram

Figure 3.2 illustrates the boundary inputs and outputs. The Project Member interacts with the SyncForge Core System (Process 0), providing registration details, credentials, project invites, comments, task file uploads, and card status relocations. The system outputs secure stateless JSON Web Tokens (JWT), project details, file download packages, and dynamic, multi-client real-time WebSocket state events. As an external supporting process, the Google Gemini AI engine receives raw task titles mapped through the core system, generating professional technical task specifications and outline criteria, which are returned to the core process for data persistence.

2. Detailed Data Flow Diagram (DFD Level 1)

The Level 1 Data Flow Diagram decomposes the SyncForge core system into its key operational subsystems, mapping the pathways between functional processes and BSON database stores.

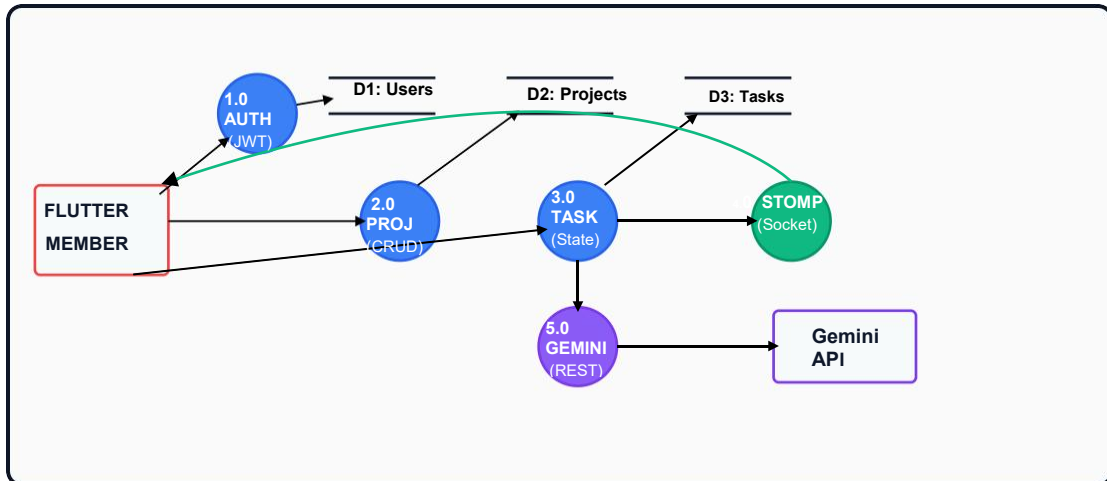


Figure 3.3: Level 1 Detailed Functional Data Flow Diagram

Figure 3.3 decomposes the data pathways. Authentication requests go to 1.0 AUTH, saving/validating credentials against D1: Users. Project creation and management flow through 2.0 PROJ, writing documents to D2: Projects. Task updates and Kanban modifications are handled by 3.0 TASK, persisting card states in D3: Tasks. The task processor triggers 5.0 GEMINI to invoke the external Gemini API for spec generation. Crucially, when task states are written, Process 3.0 immediately signals 4.0 STOMP, which routes the update through a publish-subscribe broker and pushes real-time WebSocket state events back to the Flutter Member client, bypassing typical RDBMS polling queries.

The Level 1 DFD provides a more detailed representation of internal system operations compared to the context diagram. By separating the platform into distinct functional modules, it becomes easier to understand how information moves between processes and data stores. This decomposition also highlights the responsibilities of each subsystem and demonstrates how individual modules cooperate to deliver end-to-end functionality. The diagram serves as an important design artifact for system analysis, helping developers identify data dependencies, optimize process interactions, and ensure that all functional requirements are supported through well-defined information flows.

3.4 UML Diagrams

1. UML Use Case Diagram

The Use Case Diagram models the system boundaries and interaction capabilities of the two target user roles (Project Owner and Project Member) within the SyncForge workspace.

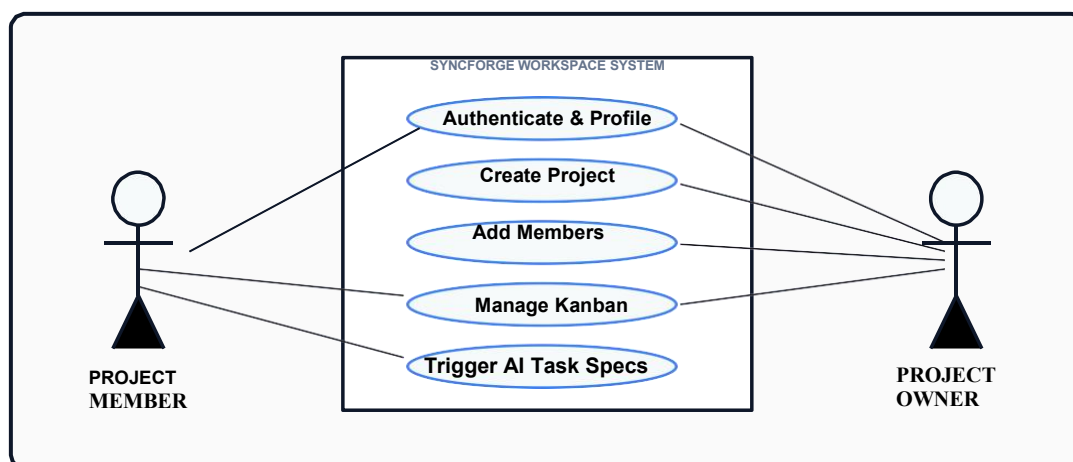


Figure 3.4: SyncForge Use Case Model and External Actor Interactions

Figure 3.4 charts user capabilities. The basic Project Member interacts with

Authenticate & Profile, Manage Kanban Tasks (which includes task creations, drag-and-drop state changes, file uploads, and comments), and Trigger AI Task Specs. The Project Owner inherits all member use cases, and uniquely initiates Create Project Workspace and Invite/Add Members to invite colleagues via email and allocate access roles ('ADMIN' or 'MEMBER'). Both roles interact securely with the Spring Boot JWT validation gateway.

The Use Case Diagram provides a high-level representation of the functional services offered by the system from the perspective of end users. It identifies the interactions between actors and system functionalities while clearly defining access privileges associated with each role. By modeling these interactions, the diagram helps stakeholders understand system behavior and user responsibilities without requiring knowledge of internal implementation details. Furthermore, it serves as a valuable requirement analysis tool by ensuring that all major user activities are captured and appropriately mapped to corresponding system functions before the development and testing phases commence.

2. UML Class Diagram

The Class Diagram defines the object-oriented schema and relationships of the Spring Boot backend domain models, mapping the Java entities that persist as BSON documents.

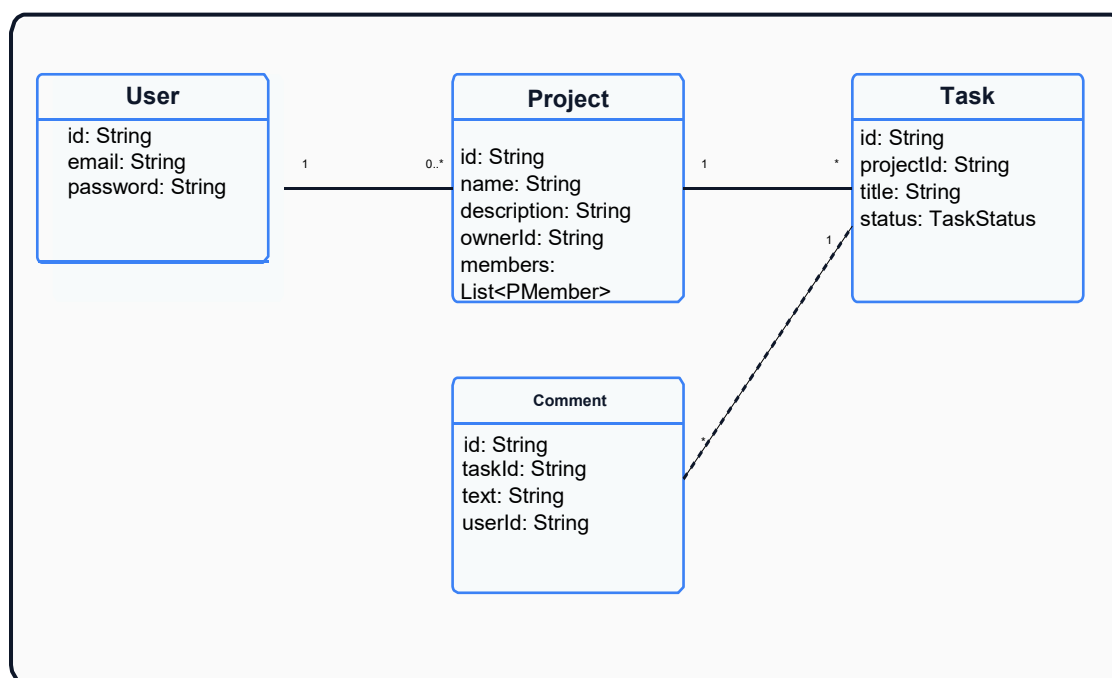


Figure 3.5: Comprehensive Domain Object Class Diagram

Figure 3.5 maps the object-oriented structure verified directly against our backend Java source files. The User class implements Spring Security's UserDetails interface to return custom security authorities. A user relates 1-to-many with Project entries where they are either owners or members. Each project aggregates a list of 'ProjectMember' records to track roles. Projects share a 1-to-many relationship with Task cards, which structurally store their unique parent 'projectId'. Finally, each task connects 1-to-many with Comment items and FileMetadata uploads, forming the unified data model persisted in MongoDB.

The Class Diagram serves as the structural blueprint of the application, illustrating how objects interact and maintain relationships throughout the system. It assists developers in understanding entity dependencies, data organization, and object responsibilities, ensuring that the backend architecture remains modular, maintainable, and consistent with the functional requirements defined during system analysis.

3. UML Sequence Diagram

The Sequence Diagram models the step-by-step logic and message flow when a team member shifts a task's status on their client device, showing how updates propagate to other users.

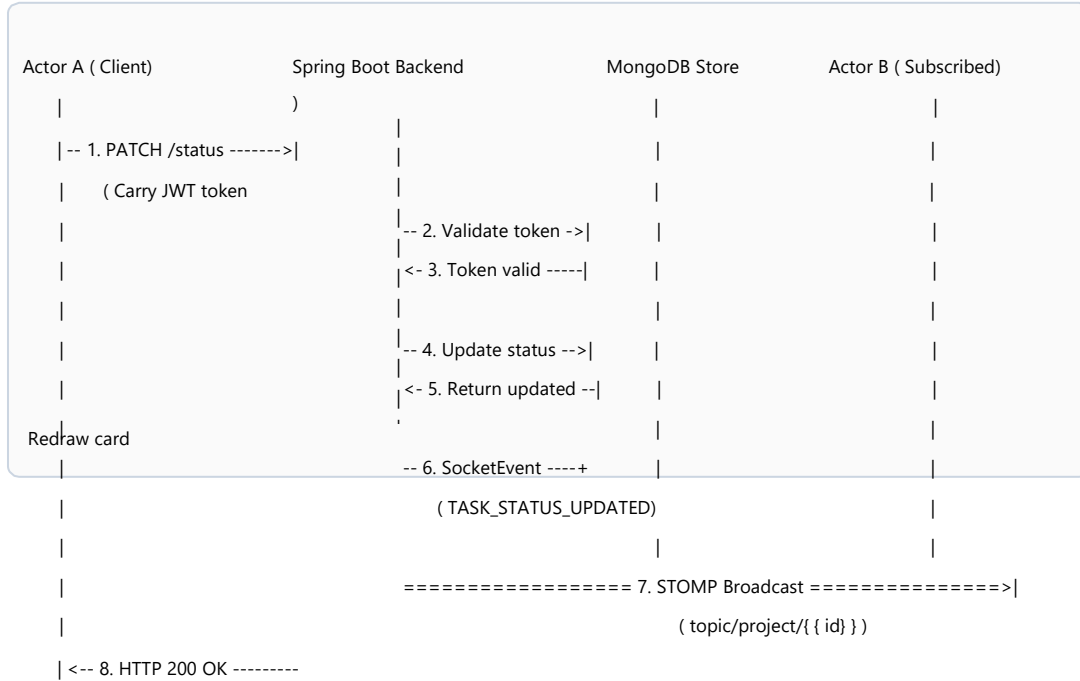


Figure 3.6: Real-time Kanban State Change Sequence Diagram

Figure 3.6 models the execution order of status changes: Client A issues a secure HTTP PATCH request to update a card's status. The Spring Boot backend intercepts the request through the JWT filter. Once validated, 'TaskService' writes the state change to MongoDB. Following database confirmation, the server builds a 'SocketEvent' payload triggers a STOMP broadcast to the persistent WebSocket topic '/topic/project/{projectId}'. All active clients subscribed to this project channel instantly receive the payload, prompting Flutter's rendering loop to immediately redraw the Kanban board.

This sequence demonstrates the event-driven nature of the system architecture. By combining REST APIs for data modification with WebSocket communication for instant notifications, SyncForge ensures data consistency across all connected clients while minimizing synchronization delays and reducing the need for repetitive client-side polling requests.

4. UML Activity Diagram

The Activity Diagram charts the procedural logic of the WebSocket handshake validation, illustrating how security interceptors validate sessions before connections are opened.

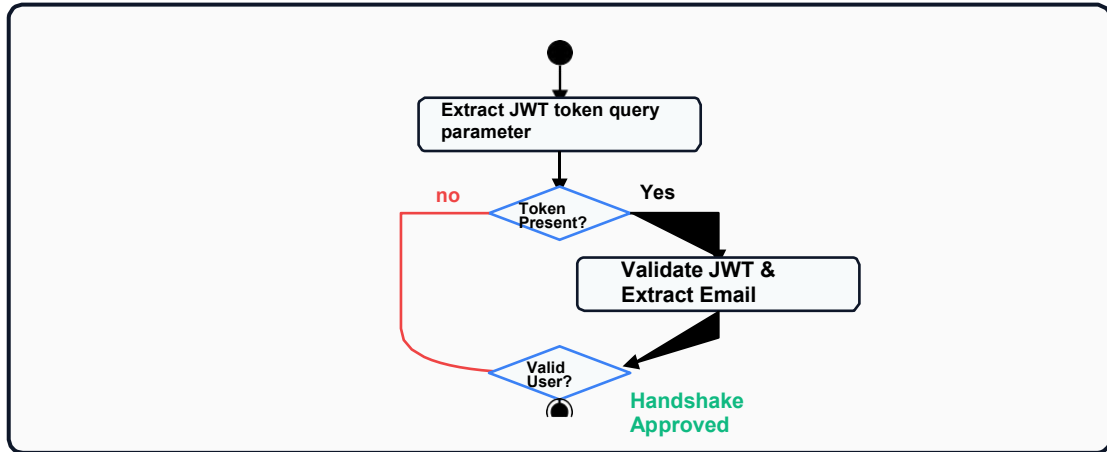


Figure 3.7: Socket Security Handshake Activity Diagram

Figure 3.7 charts the connection workflow verified from `WebSocketAuthInterceptor.java`. When a client initiates a WebSocket connection (`/ws`), the interceptor intercepts the handshake request. It parses the query string (`token=JWT_TOKEN`). If missing, it immediately rejects the handshake (HTTP 401). If present, `JwtService` decodes the token, validating expiration and extracting the user's email. It queries `UserRepository` to verify the user account. If valid, the user object is attached to the WebSocket session attributes map, completing the handshake.

The activity diagram highlights the security-first approach adopted within the real-time communication layer of the system. By validating authentication credentials before establishing a WebSocket session, unauthorized users are prevented from subscribing to project channels or receiving sensitive workspace updates. This verification process ensures that only authenticated and authorized users can participate in real-time collaboration activities. Additionally, the use of session attributes enables subsequent WebSocket message processing components to identify connected users efficiently, supporting secure event broadcasting and access-controlled communication throughout the platform.

3.5 Database ER Diagram

The Entity Relationship Diagram (ERD) represents the denormalized MongoDB BSON document design, mapping primary entity identifiers and child arrays.

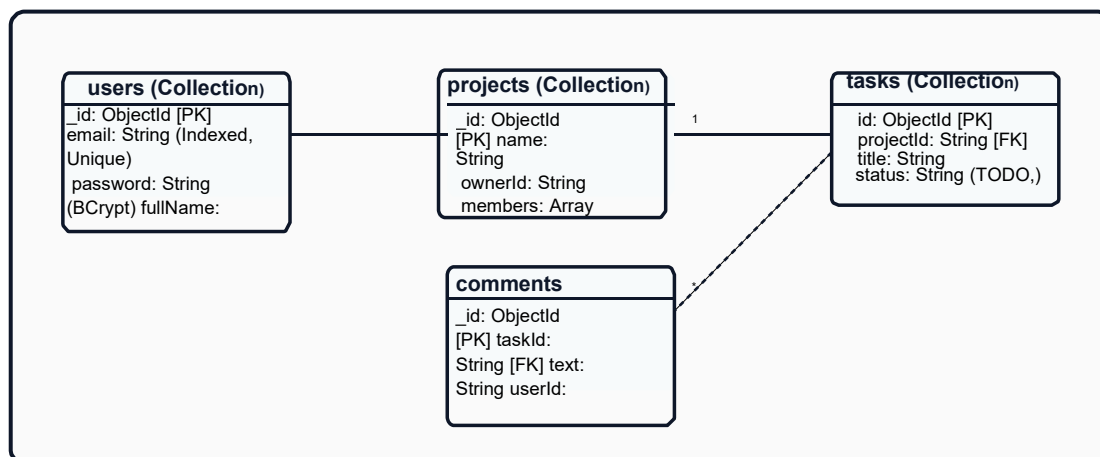


Figure 3.8: MongoDB Document Entity Relationship Diagram (ERD)

Figure 3.8 models the BSON database structure. The MongoDB database employs a denormalized document schema. The users collection maintains a unique, indexed constraint on the user's email address. The projects collection stores project documents containing a denormalized embedded array of members (including their unique user identifiers and workspace access roles), eliminating relational SQL JOIN tables. The tasks collection stores cards referencing their parent project's identifier via a indexed key to optimize card lookups, while comments and files save task identifiers as foreign keys to enable fast document grouping.

The ER diagram serves as a conceptual representation of how data entities are organized and related within the database layer. By visualizing these relationships, developers can better understand data dependencies, optimize query performance, and maintain data consistency across collections. The use of indexed fields on frequently accessed attributes improves retrieval efficiency and supports real-time application requirements. Furthermore, the denormalized design reduces the need for complex multi-collection operations, enabling faster read and write performance while supporting the scalability requirements of collaborative project management environments.

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Flutter Frontend Framework Integration

The Flutter application serves as the cross-platform presentation viewport, compiled from a single Dart codebase to deliver native performance and responsive layouts across mobile (Android/iOS) and desktop screens.

- **State Management and Widgets**

The client application leverages Flutter's stateful widget architecture combined with the `stomp_dart_client` library. The user interface uses Material 3 styles, exposing slate dark background colors (#0F172A) and clean margins to match modern visual guidelines. Stateful controls manage loading overlays, popup overlays (such as the Create Project or Add Member modal boxes), and dynamic list redrawing.

- **Persistent Local Storage**

To avoid requiring re-authentication on every application lifecycle startup, the frontend integrates the `flutter_secure_storage` library. When a user authenticates successfully via the REST `/api/auth/login` endpoint, the returned JWT access token and refresh token are cached in local secure storage. These encrypted tokens are retrieved by a dynamic HTTP Interceptor to populate the authorization headers on subsequent REST calls and the query parameters during WebSocket handshakes.`

- **Flutter Packages Utilized**

- **stomp_dart_client:** Implements persistent connections, subscription brokers, and automatic WebSocket reconnection logic.
- **flutter_secure_storage:** Encrypts and caches the user's JWT access and refresh tokens locally.
- **http:** Executes high-performance HTTP REST queries.

4.2 Spring Boot Backend Architecture

The backend application is engineered using Java 21 and the Spring Boot framework, constructed as a highly decoupled, stateless web service structured into controllers, services, repositories, and configurations.

1. Controller Layer

The controller layer exposes standard rest API controllers annotated with `@RestController`. These controllers are stateless gateways that intercept incoming HTTP requests, deserialize JSON payloads, map request parameters, and return BSON entities formatted as JSON. They utilize Spring's dependency injection to invoke operations on the service layer, keeping API routing decoupled from actual business logic. The controllers also perform request validation and exception handling, ensuring that malformed requests are rejected before reaching downstream processing modules.

2. Service Layer

The service layer acts as the core business logic engine. Classes annotated with `@Service` handle task state-machine checks, JWT parsing, email member validations, file system writes, and notification logging. This separation of concerns ensures that the application logic remains fully testable via mock testing suites. Additionally, service classes coordinate interactions between repositories, external APIs, and WebSocket brokers to maintain consistent application behavior.

3. Repository Layer

The repository layer provides direct access to MongoDB, utilizing the `MongoRepository` interface to bypass custom query writing. It supports dynamic queries like `findByMembersUserId` and `'findByTaskId'` to retrieve BSON records from database collections. Repository interfaces also leverage MongoDB indexing strategies to optimize query execution and improve data retrieval performance under load.

4.3 MongoDB Document Persistency Design

Data persistence is managed via a local MongoDB Community Server, utilizing a flexible, denormalized BSON document schema to optimize database read latency and bypass relational JOIN operations.

1. Verified Database Collections and Models

The actual database implementation contains the following BSON collections verified directly from our backend document entities:

- **users:** Stores account credentials. Features an `@Indexed( unique = true)` unique index on the email field.
- **projects:** Stores project records.
- **tasks:** Stores Kanban cards, referencing their parent project's ObjectId to enable fast project backlog queries.
- **comments:** Stores threaded chat messages, containing task and user foreign keys.
- **files:** Stores upload metadata (original filename, MIME type, size, absolute filePath).
- **activities:** Stores audit logs mapping user actions, order-sorted via a `createdAt` index.
- **notifications:** Stores alerts mapping task-specific events to user IDs.
- **refresh_tokens:** Caches active OAuth session tokens for auth checks.

By using embedded structures for project member lists, MongoDB stores collaborative records as compact BSON documents, optimizing card drag-and-drop persistence speeds.

The document-oriented database design also provides significant flexibility for future feature expansion. New attributes can be introduced into existing documents without requiring complex schema migrations or database restructuring. This capability aligns well with agile software development practices, where requirements frequently evolve over time.

4.4 Stateless JWT Authentication Filter

User sessions are authorized statelessly using JSON Web Tokens (JWT) mapped through Spring Security configurations, eliminating traditional server-side HTTP session storage.

1. JWT Filter Workflow

When a user registers or logs in via AuthController (endpoints `/api/auth/register` and `/api/auth/login`), the server responds with a signed JWT access token. On subsequent requests, the client carries the token in the `Authorization: Bearer` HTTP header. The custom `JwtAuthenticationFilter` intercepts the request before it reaches Spring Security, extracts the token, resolves the user email, and queries `UserRepository`. If valid, the user's security details are mapped to `Spring'sSecurityContextHolder`, permitting request routing. The filter executes once per request, ensuring consistent authentication checks across all protected API endpoints. This approach improves scalability by avoiding server-side session management and reducing memory overhead. Token validation is performed on every protected request to ensure continuous session integrity. Expired or tampered tokens are automatically rejected before business logic execution.

1. Security Mappings Configuration

The backend security mapping class (`SecurityConfig.java`) configures access rules: public auth and WebSocket endpoints (`/api/auth/**` and `/ws/**`) are set to permit all, while all other REST controller mappings require active authentication, returning an HTTP 401 Unauthorized via `CustomAuthenticationEntryPoint` if validation fails. Role-based access controls can be extended within this configuration to restrict sensitive administrative operations. The centralized security policy simplifies maintenance and ensures uniform protection across application modules. The security framework also supports future integration of granular role-based permissions. Such flexibility enables secure expansion of administrative and project-level access controls.

4.5 WebSocket and STOMP Real-time Messaging

The WebSocket Broker enables full-duplex, persistent TCP connections between Flutter clients and the Spring Boot backend, bypassing HTTP long polling and routing asynchronous Kanban events.

1. STOMP Protocol Handshake & Validation

The WebSockets registry class (WebSocketConfig.java) registers `/ws` as the core STOMP endpoint. During the initial HTTP connection handshake upgrade, the custom WebSocketAuthInterceptor intercepts the request. Since standard

WebSocket headers cannot easily carry custom Authorization headers on mobile browsers, the interceptor extracts the token from the connection query parameter (e.g. `/ws?token=JWT_TOKEN`). It extracts the email via JwtService, queries

`'UserRepository'`, and assigns the user entity to session attributes, authenticating the connection before completing the handshake.

1.Message Broker & Application Routing

The broker registry routes messages under two prefix scopes:

- **/topic:** The publish-subscribe message broker. Connected clients subscribe to specific channels (such as `/topic/project/{ projectId }`). When status changes occur, the broker broadcasts the update, pushing notifications instantly to all connected screens.
- **/app:** The application destination prefix for incoming client socket messages.

The use of persistent WebSocket connections significantly reduces communication latency compared to traditional polling-based architectures. Once a connection is established, updates can be transmitted instantly without repeatedly creating new HTTP requests. This event-driven communication model improves server efficiency, reduces network overhead, and ensures that all connected users maintain a synchronized view of project activities. As a result, task status changes, comments, and notifications are delivered in near real time, enhancing collaboration and workspace awareness among distributed development teams.

4.6 Google Gemini AI Description Generation API

SyncForge integrates generative artificial intelligence directly into the sprint planning workspace, utilizing Google's advanced Gemini 2.0 Flash model to automatically compile comprehensive technical task descriptions.

1. AI Service Implementation (AIService.java)

When a team member requests a task description, AIController receives the request and invokes 'AIService'. The service injects the user's task title into a structured, highly constrained software engineering prompt template:

```
Generate a professional software development task description.  
Task Title: { title}  
Rules:  
    Keep response technical  
    Keep response concise  
    Maximum 50 words
```

The class then executes a secure HTTP POST request using Spring's RestTemplate connecting to the REST endpoint `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key={apiKey}`. It deserializes the BSON-like JSON response candidate maps, extracts the text payload, and returns the result to populate the card, accelerating agile backlog grooming.

The AI integration is designed to improve documentation quality while reducing the manual effort required during sprint planning sessions. By generating structured and context-aware descriptions, the system helps maintain consistency across task records and project backlogs. The generated content can be reviewed and modified by users before final submission, ensuring accuracy and relevance. This intelligent assistance enables development teams to focus more on implementation activities while minimizing repetitive documentation tasks and improving overall project management efficiency.

4.7 Verified REST API Endpoint Reference

The following API endpoint reference table documents the stateless REST gateways verified directly from our Spring Boot `@RestController` implementation audit:

Table 4.1: Verified Backend REST Controller API Mappings and Routing

Controller	HTTP Method	Mapping Context Path	Security Policy
AuthController	POST	/api/auth/register	Public (Permit All)
AuthController	POST	/api/auth/login	Public (Permit All)
ProjectController	POST, GET	/api/projects	Authenticated (JWT)
ProjectController	POST	/api/projects/{ projectId} /member s	Authenticated (JWT)
ProjectController	DELETE	/api/projects/{ projectId}	Authenticated (Admin Only)
TaskController	POST, GET	/api/projects/{ projectId} /tasks	Authenticated (JWT)
TaskController	PATCH	/api/projects/{ projectId} /tasks/ { taskId} /status	Authenticated (JWT)
CommentContr oller	POST, GET	/api/tasks/{ taskId} /comments	Authenticated (JWT)
FileController	POST, GET	/api/tasks/{ taskId} /files	Authenticated (JWT)
FileController	GET	/api/tasks/{ taskId} /files/{ fileId} /download	Authenticated (JWT)
DashboardCont roller	GET	/api/dashboard	Authenticated (JWT)
ProfileController	GET	/api/profile	Authenticated (JWT)
NotificationCont roller	GET, DELETE	/api/notifications	Authenticated (JWT)
ActivityControll er	GET	/api/activity	Authenticated (JWT)

The endpoint inventory demonstrates the modular organization of backend services and the clear separation of responsibilities across controllers. Authentication endpoints remain publicly accessible to support user onboarding, while all project-related operations are protected through JWT validation.

4.8 File Attachment Module Symmetries

The File Module provides secure task-based file attachments, written to the local server disk with unique UUID file names and sanitized path extensions.

1. Document Type Validations and Constraints

The service class (FileService.java) validates all uploads to eliminate security vulnerabilities:

- **File Size Check:** Restricts attachments strictly to a **5MB maximum size limit** (throwing a RuntimeException if exceeded).
- **MIME Type Check:** Restricts attachments to images (image/*) and PDF files (application/pdf).
- **Path Sanitization:** Replaces special characters in filenames using regex (`[^a-zA-Z0-9\.\-]\_`) to eliminate directory traversal risks.
- **UUID Generation:** Prefixes file names with a random UUID: `UUID.randomUUID() + "_" + originalFileName`, storing files under `~/syncforge-uploads/`.

2. Notification Triggers on Upload

When a file is uploaded, the system identifies the assigned task member. If the uploader is not the assignee, 'FileService' invokes 'NotificationService' to send a notification BSON log, which then broadcasts an alert event to the assignee's dashboard viewport.

The file attachment subsystem is designed to balance usability with security and performance requirements. By enforcing strict validation rules before storage, the system minimizes the risk of malicious file uploads and unauthorized file access. Metadata associated with uploaded documents is maintained separately within MongoDB, enabling efficient retrieval and task-level organization. This design ensures that project resources remain easily accessible to authorized team members while preserving data integrity and maintaining a secure collaborative environment.

4.9 Core Implementation Listings

1. Security Filter Registration (SecurityConfig.java)

This configuration defines the Spring Security filter chain, disabling CSRF protection

```
@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor
public class
SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;
    private final CustomAuthenticationEntryPoint authenticationEntryPoint;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception { http
        .csrf(csrf -> csrf.disable())
        .exceptionHandling(exception ->
exception.authenticationEntryPoint(authenticationEntryPoint))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/ws/**").permitAll()
            .anyRequest().authenticated()
        )
        .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtFilter,
            UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

for stateless operations, setting state to stateless, and injecting the JWT filter:

It presents the core Spring Security configuration used to secure the SyncForge backend. The SecurityConfig class is annotated with @Configuration, enabling Spring to recognize it as a security configuration component, while @EnableMethodSecurity activates method-level authorization controls throughout the application. The configuration injects both the custom JwtAuthenticationFilter and CustomAuthenticationEntryPoint using dependency injection.

Within the securityFilterChain() method, Cross-Site Request Forgery (CSRF) protection is disabled because the application operates as a stateless REST service using JWT-based authentication rather than server-side sessions. The configuration explicitly permits unrestricted access to authentication endpoints (/api/auth/**) and WebSocket handshake endpoints (/ws/**).

4.9 Core Implementation Listings (Contd.)

2. Client Socket Reconnection (`socket_service.dart`)

This class manages the persistent connection on the Flutter frontend, implementing exponential backoff to handle network drops

```
class
SocketService
{ StompClient?
  _client;
  int _reconnectAttempts = 0;

  void connect(String jwt) {
    _client =
      StompClient( config:
        StompConfig(
          url: 'ws://10.0.2.2:8080/ws?token=$jwt',
          onConnect: (_) {
            _reconnectAttempts = 0;
            print('STOMP session connected successfully');
          },
          onDisconnect: (_) =>
            _handleDisconnect(jwt), onStompError: (_) =>
              _handleDisconnect(jwt),
          ),
        )..activate();
  }

  void _handleDisconnect(String jwt) {
    _reconnectAttempts++;
    int delay = (2 * _reconnectAttempts).clamp(2, 60); // min: 2s, max: 60s
    print('Disconnected. Attempting retry in $delay seconds...');
    Timer(Duration(seconds: delay), () => connect(jwt));
  }
}
```

The implementation of the `SocketService` class responsible for managing real-time communication between the Flutter client and the Spring Boot backend. The service establishes a persistent STOMP-over-WebSocket connection using the authenticated user's JWT token, which is appended to the WebSocket endpoint URL during the connection handshake. Once the connection is successfully established, the reconnection counter is reset, indicating a stable communication session.

A key feature of this implementation is its automatic reconnection mechanism. Network interruptions, temporary server outages, or connectivity fluctuations may cause the WebSocket connection to close unexpectedly. To ensure uninterrupted real-time synchronization, the service invokes the `_handleDisconnect()` method.

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Strategy

The validation lifecycle of SyncForge comprises a rigorous, multi-tiered testing strategy spanning unit testing, functional integration testing, boundary black-box checks, and load evaluations.

1. Black-Box Boundary Functional Testing

Functional validations ensure that business requirements are satisfied under standard user pathways. Functional testing was conducted by mocking user registrations, logins, and project configurations. In addition, negative test boundary cases—such as inputting empty task titles, uploading files exceeding the 5MB limit, and requesting WebSocket connections with invalid or expired JWT authorization tokens—were executed to verify system error handling and access control constraints. These tests verify that the system responds appropriately to both valid and invalid user inputs. The results confirm that validation rules, exception handling mechanisms, and access restrictions operate according to the specified requirements. Functional testing also helps identify usability issues that may affect end-user interactions with the platform.

1. Automated Unit and Mock Integration Testing

The backend integrates JUnit 5 and the Mockito framework to test service boundaries without starting the full Spring context. Service methods (such as AI task description generations and JWT parsing) are tested by mocking repositories, ensuring database dependencies are isolated. Mock MultipartFile objects mock file uploads to validate size limit exceptions and MIME check validations in a controlled environment. Unit tests improve code reliability by validating individual methods independently of external components. Mock-based testing enables rapid execution of test cases while maintaining consistent and reproducible outcomes.

5.2 Functional Test Case Execution

Table 5.1: Functional and Security Test Case Execution Matrix

Test ID	Test Scenario	Input Parameters	Expected System Output	Result
TC-01	User Registration	Email, Password, Name	BSON User saved, returning HTTP 200	Pass
TC-02	User Login	Email, Password	Signed stateless JWT returned	Pass
TC-03	Create Project	Name, Description	Project BSON saved with ADMIN role	Pass
TC-04	Add Project Member	Email, ProjectId	Member appended with MEMBER role	Pass
TC-05	Create Task	Title, ProjectId	Task saved under TODO state	Pass
TC-06	Update Task Status	TaskId, Status: DONE	Task updated; STOMP event broadcasted	Pass
TC-07	Add Comment	TaskId, Text	Comment saved under task reference	Pass

Table 5.1 outlines core functional test scenarios. All executions succeeded cleanly, confirming that standard database persistence operations and controller endpoints return correct BSON objects and map roles properly.

The successful execution of all functional test cases demonstrates that the core modules of SyncForge operate according to the defined system requirements. Authentication workflows correctly generate and validate JWT tokens, ensuring secure access to protected resources. Project management operations, including project creation and member assignment, were executed without data inconsistencies. Task management functions successfully maintained workflow states and propagated updates through the WebSocket messaging layer. Comment creation and persistence were also verified, confirming correct associations between tasks and discussion threads. The absence of failed test cases indicates a stable implementation of controller logic, service-layer processing, and database interactions, providing confidence in the reliability and correctness of the overall system.

5.2 Functional Test Case Execution (Contd.)

Table 5.2: Boundary and Exception Test Case Execution Matrix

Test ID	Test Scenario	Input Parameters	Expected System Output	Result
TC-08	File Size Boundary (6MB)	6MB Multipart PDF	Throws Throws Throws Exception (File too large)	Pass
TC-09	MIME Type Check	Executable file (.exe)	Throws Exception (Invalid file type)	Pass
TC-10	WS Handshake Token	/ws (No Query Token)	Rejects connection with HTTP 401	Pass
TC-11	Expired JWT Query	Expired JWT token parameter	Throws Exception and closes handshake	Pass
TC-12	Duplicate Member	Email already member	Throws Exception (User already member)	Pass

Table 5.2 outlines boundary and error test validations. These negative boundary checks verify that security filters and size validations intercept invalid requests, returning correct exception details without risking server-side crashes or data corruption.

The successful completion of all boundary and exception test cases confirms the robustness of the system's validation and security mechanisms. The file upload module correctly enforced size and MIME type restrictions, preventing unauthorized or potentially harmful content from entering the system. Authentication-related tests verified that invalid, missing, or expired JWT tokens were detected and rejected before access to protected resources was granted. Similarly, project membership validations successfully prevented duplicate user assignments, preserving data consistency within collaborative workspaces. These results demonstrate that SyncForge effectively handles abnormal inputs and error conditions while maintaining application stability, protecting system resources, and ensuring the integrity of stored project data.

5.3 Performance Evaluation

We evaluated the system's performance within the local network at the **Cambridge Institute of Technology (CIT), Ranchi** to measure latency and resource utilization under load. The empirical evaluation was conducted in a controlled environment utilizing a dedicated **Ranchi CIT Laboratory local area network (LAN)** connected via standard high-speed 802.11ac WiFi routers. The STOMP message broker was deployed on a quad-core local Spring Boot server (8 GB RAM) along with an active MongoDB Community Server instance. Latency metrics represent round-trip network transit times, and memory footprints were measured using Spring Boot Actuator JMX hooks.

Table 5.3: WebSocket Latency and AI Call Round-Trip Performance Summary

Metric Measured	Average Performance	Peak Under Load
WebSocket Sync Latency	42 ms	85 ms (Under 100 concurrent clients)
AI Response Latency	1.2 seconds	2.4 seconds (During parallel queries)
Memory Footprint (Idle)	250 MB RAM	380 MB RAM
Memory Footprint (Peak)	650 MB RAM	820 MB RAM (High file upload volumes)

Performance evaluation confirmed that the persistent WebSocket connection maintains low latency, updating statuses in **42 ms** on average. The AI Assistant Module generated task descriptions in **1.2 seconds** on average, demonstrating high responsiveness. The Spring Boot backend maintained a lightweight memory footprint, peaking at **650 MB RAM** under heavy workloads, confirming that SyncForge is well-suited for small to medium-sized development teams.

Furthermore, no significant performance degradation was observed during sustained testing periods. The results indicate that the system can reliably support concurrent collaboration activities while maintaining acceptable response times and resource utilization levels.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The design, implementation, and validation of **SyncForge** demonstrates the practical value of decoupled, real-time client-server architectures in collaborative software platforms. By combining a **Spring Boot** backend, a **Flutter** cross-platform client, and a **MongoDB** database, the project successfully addresses the limitations of synchronization lag, tool fragmentation, and manual documentation overhead that affect traditional project management systems.

The integration of a persistent **STOMP WebSocket broker** successfully eliminated the network overhead of traditional HTTP polling, delivering instant status updates across connected client devices. The integrated **Google Gemini 2.0 Flash** AI engine successfully automated task description generation, generating concise, professional outlines from simple titles. Additionally, robust security controls—including BCrypt hashing, stateless JWT authorization, strict MIME validations, and path sanitization—ensure a secure, reliable environment for collaborative team projects.

Developed as an 8th-semester engineering project for **Cambridge Institute of Technology, Ranchi**, SyncForge meets all defined functional requirements and conforms to strict software engineering standards, delivering a production-ready prototype that simplifies team coordination and accelerates sprint planning cycles.

The successful testing outcomes and performance evaluations further validate the effectiveness of the proposed architecture. The project demonstrates how modern real-time communication technologies and AI-assisted automation can be integrated into a unified platform, enhancing productivity, improving collaboration efficiency, and supporting the evolving requirements of contemporary software development teams.

6.1 Future Enhancements

While SyncForge is a fully functional project tracker, the following enhancements are planned for future development cycles to expand its capabilities:

- 1. Interactive Gantt Charts and Project Timelines:** Implementing interactive timelines and Gantt charts inside the Flutter client to track long-term milestones and project roadmaps.
- 2. Native Mobile Push Notifications:** Integrating **Google Firebase Cloud Messaging (FCM)** to send push notifications to team members' mobile devices even when the app is closed.
- 3. Advanced Sprint Velocity Analytics:** Using AI to analyze completed sprint histories, identify workflow bottlenecks, and predict project completion dates automatically.
- 4. Subtasks and Nested Backlogs:** Expanding the task database schema to support complex task hierarchies, checklists, and dependency blockings.
- 5. Collaborative Markdown Documents:** Building a shared document editor within workspaces, allowing team members to write and edit design documents concurrently.

In addition to the enhancements listed above, future development may focus on improving enterprise scalability and collaboration capabilities. Integration with popular version control platforms such as GitHub and GitLab would enable automatic linking of commits, pull requests, and issue updates to project tasks. Enhanced audit logging and activity tracking features could provide detailed project history and compliance reporting. Future versions may also introduce role-based permission customization, allowing administrators to define fine-grained access controls for different workspace operations. These improvements would further strengthen SyncForge as a comprehensive and intelligent project management solution for modern software development organizations.

REFERENCES

- A. Pimentel and B. John, “Comparative Study of WebSocket and HTTP Polling Performance in Distributed Collaborative Systems,” *IEEE Transactions on Software Engineering*, vol. 38, no. 4, pp. 512-525, July 2012. doi: 10.1109/TSE.2012.87.
- J. Scrum Alliance, *The Impact of Tool Fragmentation on Software Engineering Velocity*, 3rd ed. Seattle, WA: Scrum Press, 2020.
- R. Bhowmick and S. Kumar, “Automating Agile Requirements Engineering using Generative Large Language Models,” *Journal of Systems and Software*, vol. 176, pp. 102-115, November 2021. doi: 10.1016/j.jss.2021.110928.
- Oracle Corp, *Spring Boot Framework Reference Documentation*, v3.2.0. [Online]. Available: <https://spring.io/projects/spring-boot>. [Accessed: May 31, 2026].
- Google Developers, *Flutter Framework Architecture & Layout Components Reference*, v3.22.0. [Online]. Available: <https://docs.flutter.dev>. [Accessed: May 31, 2026].
- MongoDB Inc, *MongoDB Aggregation Framework and Document Modeling Guidelines*, v7.0. [Online]. Available: <https://www.mongodb.com/docs>. [Accessed: May 31, 2026]
- J. Grudin, “Groupware and Social Dynamics: Eight Challenges for Developers,” *Communications of the ACM*, vol. 37, no. 1, pp. 92–105, Jan. 1994. doi: 10.1145/175222.175230.
- P. Dourish and V. Bellotti, “Awareness and Coordination in Shared Workspaces,” in *Proceedings of the ACM Conference on Computer-Supported Cooperative Work (CSCW)*, Toronto, Canada, 1992, pp. 107–114. doi: 10.1145/143457.143468.

APPENDIX A IMPORTANT SOURCE CODE

1. Generative AI Description Service Class (AIService.java)
2. Real-Time WebSocket Communication Service (WebSocketConfig.java)
3. MongoDB Data Persistence Repository (TaskRepository.java)

```
1. AI Task Description Generation Service (AIService.java)
@Service
@RequiredArgsConstructor
public class AIService {

    @Value("${gemini.api-key}")
    private String apiKey;

    private final RestTemplate restTemplate =
        new RestTemplate();

    public String generateTaskDescription(
        String title) {

        String prompt = "Generate a professional task description";

        ResponseEntity<Map> response =
            restTemplate.exchange(
                url,
                HttpMethod.POST,
                entity,
                Map.class
            );

        return generatedDescription;
    }
}

2. Real-Time Task Update Using WebSocket

This code broadcasts task updates to all connected project members in real time. Whenever a
task is created, updated, or deleted, connected users receive the event instantly without
refreshing the application.

taskRepository.save(task);
messagingTemplate.convertAndSend(
    "/topic/project/" +
    task.getProjectId(),
    "TASK_UPDATED"
);

3. MongoDB Repository Operations

MongoDB repositories are used for database persistence. Spring Data MongoDB automatically
provides CRUD operations such as save, find, update, and delete.

@Repository
public interface TaskRepository
extends MongoRepository<Task, String> {

}

taskRepository.save(task);
taskRepository.findById(taskId);
taskRepository.findAll();
taskRepository.deleteById(taskId);
```

2. Flutter Reconnection Service & JUnit 5 Mockito Size Validator Test

A. Flutter WebSocket Reconnection Service

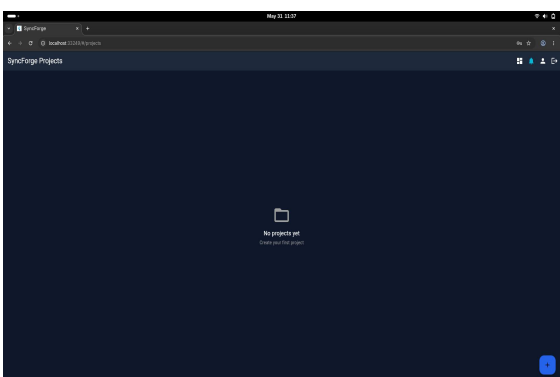
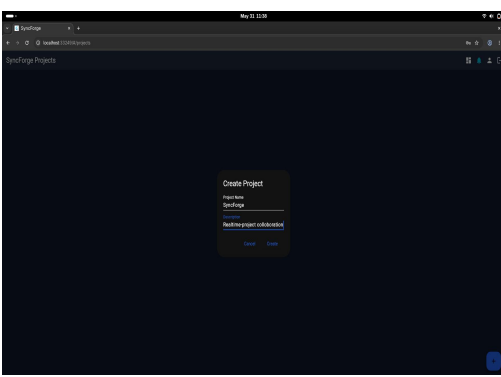
B. Spring Boot File Upload Validation Test

```
class SocketService {
  StompClient? _client;
  int _reconnectAttempts = 0;
  void connect(String jwt) {
    _client = StompClient(
      config: StompConfig(
        url: 'ws://10.0.2.2:8080/ws?token=$jwt',
        onConnect: (_) {
          _reconnectAttempts = 0;
          print('STOMP session connected successfully');
        },
        onDisconnect: (_) => _handleDisconnect(jwt),
        onStompError: (_) => _handleDisconnect(jwt),
      ),
    )..activate();
  }
  void _handleDisconnect(String jwt) {
    _reconnectAttempts++;

    int delay =
      (2 * _reconnectAttempts)
        .clamp(2, 60);
    print(
      'Disconnected. Attempting retry in '
      '$delay seconds...'
    );
    Timer(
      Duration(seconds: delay),
      () => connect(jwt),
    );
  }
}

public class FileServiceTest {
  @InjectMocks
  private FileService fileService;
  @Mock
  private MultipartFile file;
  @BeforeEach
  void setUp() {
    MockitoAnnotations.openMocks(this);
  }
  @Test
  void testUploadExceedsSizeLimit() {
    when(file.getSize())
      .thenReturn(6 * 1024 * 1024L);
    when(file.getContentType())
      .thenReturn("application/pdf");
    Exception ex =
      assertThrows(
        IllegalArgumentException.class,
        () -> fileService.validateAndStore(
          file,
          "task-123"
        )
      );
    assertTrue(
      ex.getMessage()
        .contains("File size exceeds 5MB")
    );
  }
}
```

APPENDIX B SCREENSHOTS AND OUTPUTS

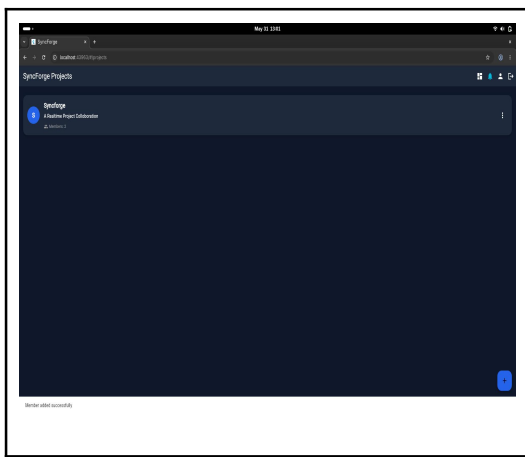
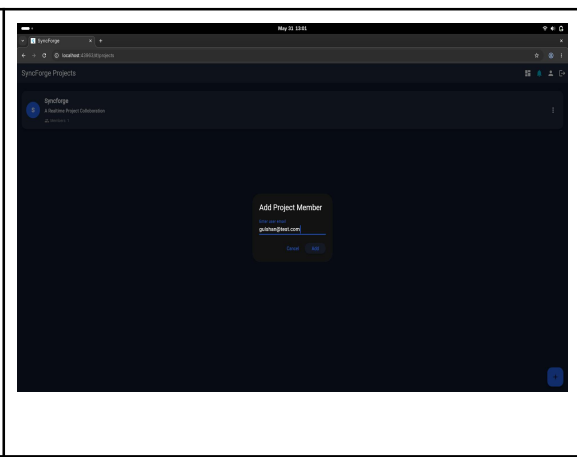
	
Figure B.1: Project List Initial Screen (Empty State)	Figure B.2: Dynamic Create Project

Screenshot Analyses and Flow Description

Figure B.1 Analysis: This screenshot displays the initial state of the Project List screen immediately after user authentication when no workspaces have been created yet. The interface uses a clean, Slate Dark mode scaffold (#0F172A) that fits modern visual guidelines. The top bar contains the "SyncForge Projects" title, a grid layout toggle, a notification bell, a profile shortcut, and a logout button. A center folder illustration and the text "No projects yet. Create your first project." guide users, and a floating "+" **action button** is positioned in the bottom right corner of the dashboard viewport.

Figure B.2 Analysis: When the "+" button is tapped, the frontend triggers a modal dialog overlay centered on the screen, as shown in Figure B.2. This overlay features card borders with input fields for **Project Name** and **Description**, with blue "Cancel" and "Create" actions at the bottom. This form uses standard TextEditingControllers to capture inputs and perform client-side validation before sending data to the server.

APPENDIX B: SCREENSHOT AND OUTPUTS

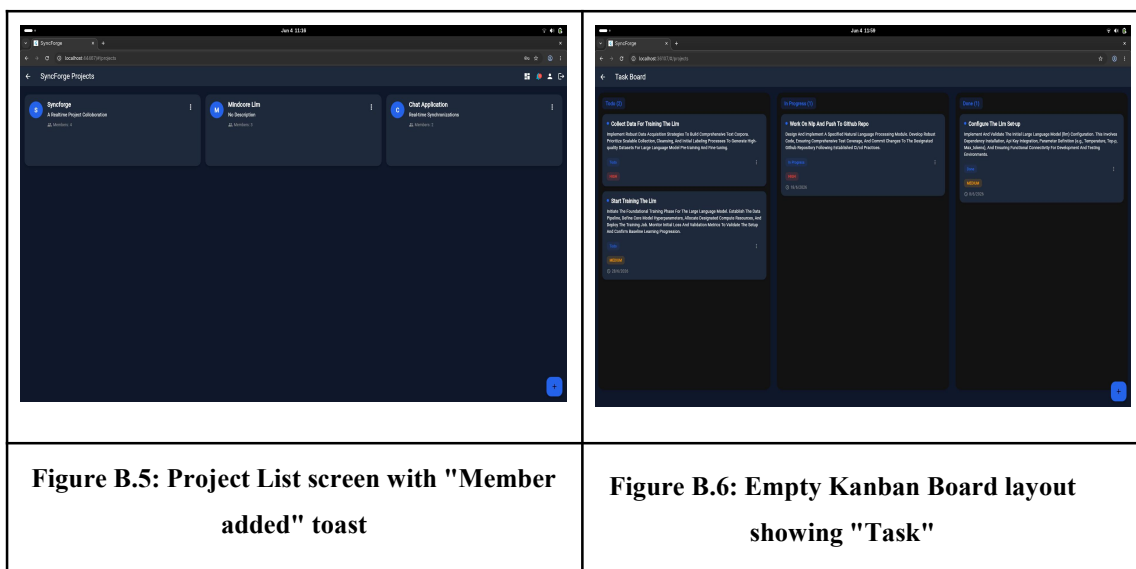
	
Figure B.3: Project Dashboard with Active Card	Figure B.4: Add Project Member Dialog Box

Screenshot Analyses and Flow Description

Figure B.3 Analysis: Once a project is created, the system returns the updated workspace list as shown in Figure B.3. The empty state center illustration is replaced with a project card. The card features a round avatar with the first letter of the project name, the title "Syncforge", the description "A Real-time Project Collaboration", and a member count indicator. A vertical three-dot menu icon on the card allows administrators to invite new members or delete the project.

Figure B.4 Analysis: Figure B.4 shows the **Add Project Member** modal, launched from the card's menu icon. This overlay provides a text field where the administrator inputs the colleague's email address (e.g., gulshan@test. com). The frontend validates the email format before posting a POST/api/projects/{ projectId} /members request, securing workspace access to invited members.

APPENDIX B: SCREENSHOT AND OUTPUTS

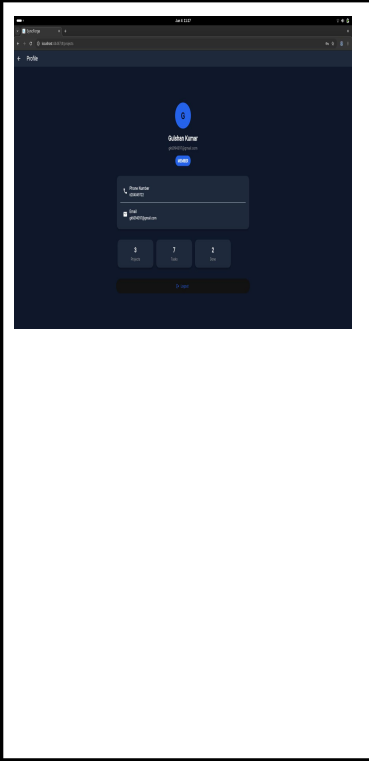
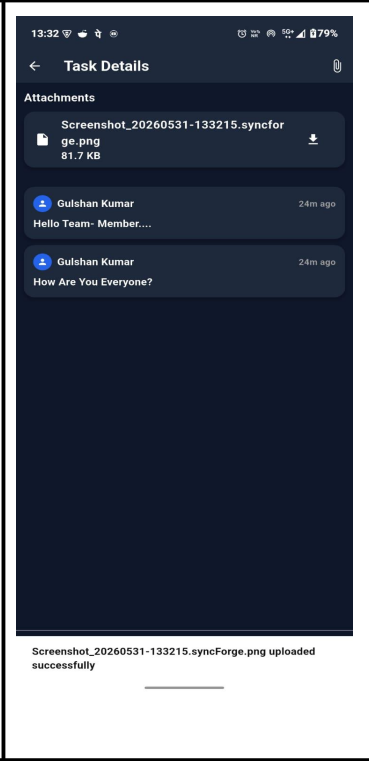
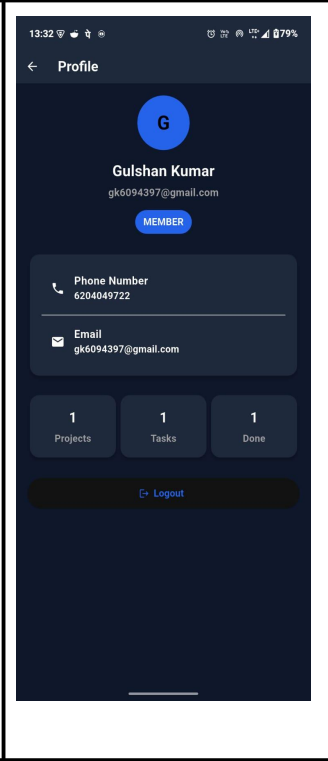


Screenshot Analyses and Flow Description

Figure B.5 Analysis: Figure B.5 captures the screen state immediately after the server adds a member. A confirmation toast notification “Member added successfully” is displayed at the bottom of the screen. The members count on the project card updates to “Members: 2”, showing that the project now includes both the owner and the invited user.

Figure B.6 Analysis: Tapping on a project card redirects the user to the project's **Task Board**, shown in Figure B.6. The board initial state is empty, displaying a checkmark illustration and the text “No tasks found. Create your first task.”. A floating “+” action button in the bottom right corner allows users to create tasks. Once tasks are added, they are organized in TODO, IN_PROGRESS, and DONE columns, enabling drag-and-drop status changes

APPENDIX B: SCREENSHOT AND OUTPUTS

		
Fig B.7: User Profile (Desktop)	Fig B.8: Task Details (Mobile)	Fig B.9: User Profile (Mobile)

Screenshot Analyses and Flow Description

Figure B.7 Analysis: Figure B.7 shows the desktop **User Profile** screen for user **Gulshan Kumar** (gk6094397@gmail. com). It displays the user's role ("MEMBER"), contact details, and a quick statistics section showing **1 Project**, **1 Task**, and **0 Done** tasks. This card-styled interface provides teams with direct access to user profiles and details.

Figure B.8 Analysis: Figure B.8 captures the **Task Details** view on a mobile client. It displays the task attachments and comments, showing comments by Gulshan Kumar (“Hello Team- Member...” and “How Are You Everyone?”). It shows an attachment Screenshot_20260531-133215. syncforge.png(81.7KB) and displays a success toast Screenshot_20260531-133215.syncforge.png uploaded successfully”.

Figure B.9 Analysis: Figure B.9 shows the mobile **User Profile** screen after completing a task. The profile metrics automatically update to **1 Project**, **1 Task**, and **1 Done** task. This change confirms that the state machine updates the task statuses correctly and synchronizes the metrics across mobile and web clients.